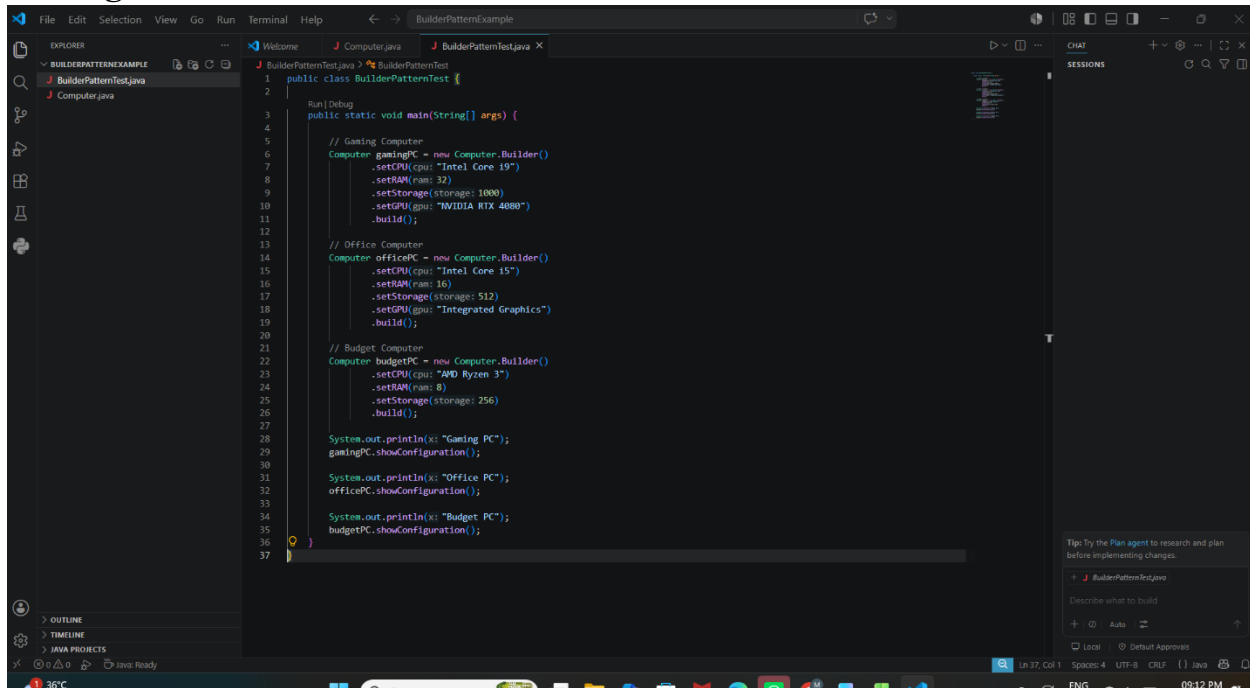


Hands – On Report

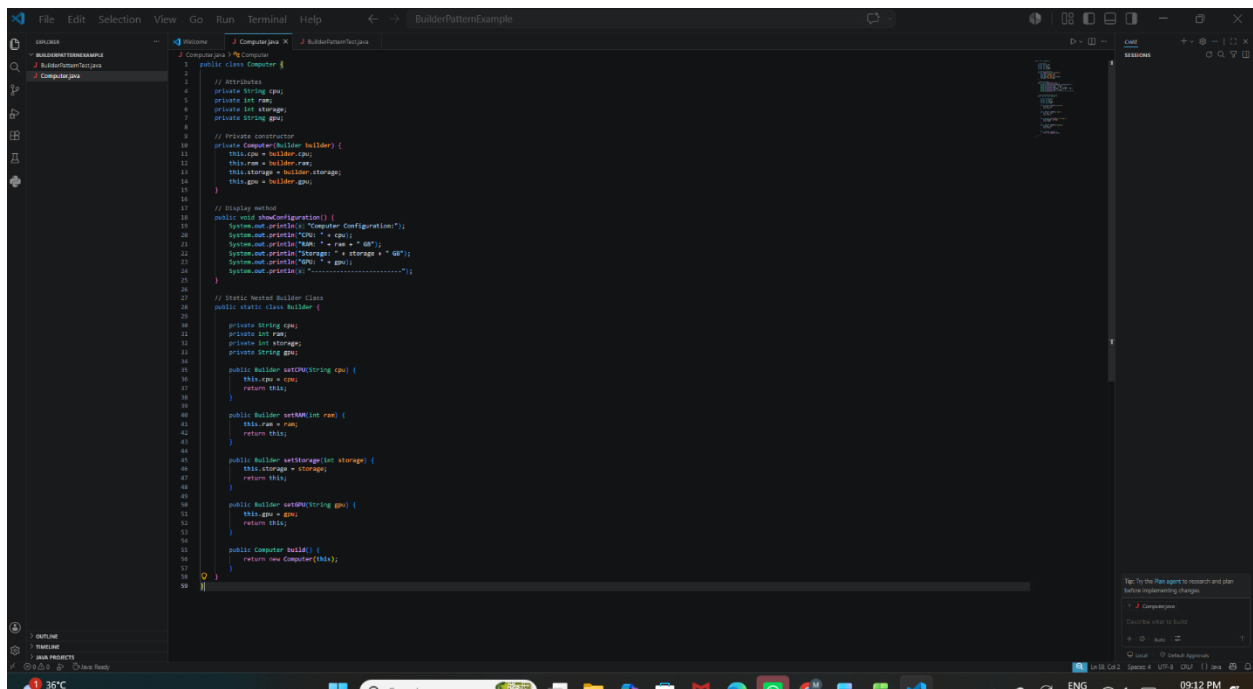
Exercise 3: Implementing the Builder Pattern

In this module, I was able to create, execute and test the Builder Pattern in java using a project called 'BuilderPatternExample'. I designed a Computer class with attributes like CPU, RAM, Storage, and created a static nested ComputerBuilder class with methods to set each of these attributes, and a build() method to build Computer objects. I also had a private constructor in the Computer class which took a Builder object to make sure that objects are created in a controlled manner. Once the program was implemented, I executed it and had various combinations of Computer objects to ensure that the Builder Pattern works. This practical activity explained to me the use of the Builder Pattern to make complex objects easier to create with optional parameters, and to make the code easier to read, flexible, and maintain.

Coding:



```
1 public class BuilderPatternTest {
2
3     public static void main(String[] args) {
4
5         // Gaming Computer
6         Computer gamingPC = new Computer.Builder()
7             .setCPU(cpu: "Intel Core i9")
8             .setRAM(ram: 32)
9             .setStorage(storage: 1000)
10            .setGPU(gpu: "NVIDIA RTX 4080")
11            .build();
12
13        // Office Computer
14        Computer officePC = new Computer.Builder()
15            .setCPU(cpu: "Intel Core i5")
16            .setRAM(ram: 16)
17            .setStorage(storage: 512)
18            .setGPU(gpu: "Integrated Graphics")
19            .build();
20
21        // Budget Computer
22        Computer budgetPC = new Computer.Builder()
23            .setCPU(cpu: "AMD Ryzen 3")
24            .setRAM(ram: 8)
25            .setStorage(storage: 256)
26            .build();
27
28        System.out.println(x: "Gaming PC");
29        gamingPC.showConfiguration();
30
31        System.out.println(x: "Office PC");
32        officePC.showConfiguration();
33
34        System.out.println(x: "Budget PC");
35        budgetPC.showConfiguration();
36
37    }
```



Output:

